

1. A genealogia das linguagens não é uma escada de progresso.

Explique essa afirmação e apresente dois fatores históricos que fazem uma linguagem influenciar outra sem necessariamente substituí-la.

Objetivos: obj01, obj05 · Referência: Sebesta, cap. 2, páginas PDF 50, 51.

A afirmação está dizendo que a evolução das linguagens de programação não segue uma linha reta e hierárquica em que cada linguagem nova é "melhor" e "substitui" a anterior, como se fossem degraus de uma escada em direção a um topo. Em vez disso, a genealogia é mais parecida com uma árvore (ou uma rede) de influências: linguagens antigas continuam vivas, sendo usadas, mantidas e evoluídas em paralelo com o surgimento de novas linguagens, e uma linguagem pode influenciar profundamente outras sem que isso signifique que ela se tornou obsoleta ou foi "superada".

1. Especialização para domínios diferentes

Cada linguagem costuma nascer para resolver um tipo específico de problema (computação científica, processamento de dados comerciais, inteligência artificial, programação de sistemas etc.)

2. Continuidade institucional e investimento acumulado (legado)

Organizações, governos e comunidades inteiras investem tempo, dinheiro e treinamento em uma linguagem, código legado, compiladores, programadores treinados, sistemas críticos já em produção. Isso cria uma inércia que mantém a linguagem viva e em evolução (recebendo novas versões, como Fortran 2003 evoluindo a partir do Fortran I) mesmo que suas ideias tenham sido "emprestadas" e refinadas por linguagens mais recentes.

2. Plankalkül não foi implementada em sua época. Ainda assim, por que ela é relevante para a história das linguagens? Cite três recursos antecipados por seu projeto e explique o valor de um deles.

Objetivos: obj01, obj02 · Referência: Sebesta, cap. 2, páginas PDF 52, 53.

Mesmo nunca tendo sido implementada, Plankalkül é relevante porque seu projeto, concebido por Konrad Zuse já em 1945, antecipou conceitos que só apareceriam em outras linguagens décadas depois, quando o manuscrito foi finalmente publicado em 1972, muitas dessas ideias já haviam sido "reinventadas" independentemente por outras linguagens, mas isso mostra o quão avançado era o pensamento de Zuse para sua época. Ela é, assim, um marco teórico na história da área, mesmo sem impacto prático direto e imediato.

Estruturas de dados avançadas: Plankalkül já incluía vetores e registros (equivalentes aos structs de linguagens baseadas em C), com a possibilidade de registros aninhados, além de tipos escalares construídos a partir do tipo bit,

incluindo um esquema de ponto flutuante com notação de complemento de dois e "bit oculto".

Sentença iterativa (semelhante ao for): apesar de não possuir um comando de desvio incondicional (goto) explícito, a linguagem incluía uma construção iterativa similar ao for de Ada, além de um comando (Fin) para sair de aninhamentos de laços ou reiniciar um ciclo iterativo.

Expressões matemáticas como asserções: um dos recursos mais interessantes dos programas de Zuse era a inclusão de expressões matemáticas que documentavam os relacionamentos válidos entre variáveis em determinados pontos do código, informando o que deveria ser verdadeiro durante a execução naquele ponto.

3. Compare Short Code, Speedcoding e os sistemas A-0/A-1/A-2 quanto ao problema enfrentado e à estratégia adotada. Por que chamá-los simplesmente de compiladores modernos seria impreciso?

Esses sistemas enfrentavam o mesmo problema fundamental: a extrema dificuldade, lentidão e alta propensão a erros da programação direta em código de máquina absoluto nos computadores da década de 1950, somada à ausência de hardware dedicado para operações com ponto flutuante (*floating-point*) e indexação de memória.

Quanto à estratégia adotada:

- Short Code (1949): Desenvolvido por John Mauchly para o BINAC (e depois UNIVAC), utilizava pseudocódigos interpretados. O código era traduzido instrução por instrução em tempo de execução, permitindo usar expressões matemáticas mais legíveis, mas com um custo enorme de desempenho.
- Speedcoding (1954): Criado por John Backus para o IBM 701, simulava um "computador virtual" dentro do hardware existente por meio de um sistema interpretativo, oferecendo instruções de ponto flutuante e auto-incremento de índices.
- Sistemas A-0/A-1/A-2 (1951-1953): Desenvolvidos por Grace Hopper na Remington Rand, usavam um sistema primitivo de expansão de pseudo-código. O sistema buscava rotinas pré-compiladas guardadas em fita magnética a partir de códigos curtos de chamada e as concatenava, gerando o programa em linguagem de máquina.

Chamá-los simplesmente de compiladores modernos é impreciso porque a maioria deles operava puramente por interpretação (sendo até 50 vezes mais lentos que o código nativo) ou por simples macro-expansão e substituição de código. Eles não realizavam análise sintática e semântica complexas, não faziam otimização de código e não traduziam expressões algébricas de forma autônoma como um compilador moderno faz.

4. Explique por que o projeto Fortran precisou convencer programadores de que código traduzido podia competir com código de máquina escrito à mão. Relacione desempenho, custo de programação e adoção.

Nos anos 1950, o tempo de processador e a memória dos computadores eram escassos e caríssimos. Os programadores da época eram especialistas altamente treinados para extrair o máximo de desempenho escrevendo instruções de máquina manuais. Se o Fortran gerasse um código traduzido 20% ou 30% mais lento ou mais pesado que o código feito à mão, a comunidade rejeitaria a linguagem imediatamente, pois o custo financeiro do tempo de computação desperdiçado superaria qualquer ganho de tempo de desenvolvimento.

A relação entre desempenho, custo de programação e adoção se deu da seguinte forma:

- Desempenho: Para garantir a adoção inicial, a equipe de John Backus focou o projeto do Fortran I quase inteiramente na otimização de código, provando que o compilador conseguia gerar instruções em linguagem de máquina com velocidade comparável (ou até superior) à de um programador humano.
- Custo de programação: Escrever equações matemáticas em notação algébrica no Fortran reduziu o tempo e o custo de desenvolvimento em até 20 vezes em comparação com a escrita manual em assembly ou código de máquina.
- Adoção: Uma vez demonstrado que a velocidade de execução não era sacrificada, o mercado e a comunidade científica adotaram o Fortran em massa, pois a economia no tempo do programador passou a valer a pena sem penalizar o custo do hardware.

5. Lisp surgiu em um contexto diferente de Fortran. Compare os domínios, a representação de dados e o estilo de computação favorecido pelas duas linguagens.

Fortran e Lisp surgiram para resolver problemas opostos com abordagens totalmente distintas:

- Domínio principal: Fortran foi projetado para computação científica e processamento numérico-algébrico intensivo. Lisp foi desenhado para pesquisas em Inteligência Artificial e processamento de símbolos não numéricos.
- Representação de dados: Fortran utiliza estruturas rígidas, fixas e homogêneas, focando em escalares, vetores e matrizes multidimensionais. Lisp representa dados e o próprio código de forma dinâmica e flexível por meio de listas encadeadas e átomos (as chamadas *S-expressions*).
- Estilo de computação: Fortran favorece o estilo imperativo/procedural, baseado em alteração de estados, atribuições explícitas e laços de repetição

(*loops*). Lisp favorece o estilo funcional, baseado na aplicação de funções, avaliação de expressões, recursão e minimização de efeitos colaterais.

6. Avalie três contribuições de ALGOL 60 que ultrapassaram sua adoção comercial. Por que uma linguagem pode ser muito influente sem dominar o mercado?

Três contribuições fundamentais do ALGOL 60 que moldaram a ciência da computação:

1. Sintaxe formal com BNF (Backus-Naur Form): Foi a primeira linguagem a ter sua sintaxe descrita de forma rigorosa e matemática por meio de uma gramática livre de contexto, criando o padrão até hoje usado para definir linguagens.
2. Estrutura de blocos e escopo de variáveis: Introduziu os delimitadores **begin** e **end** para aninhamento de blocos e formalizou os conceitos de variáveis locais e globais e seus tempos de vida.
3. Recursão e passagem de parâmetros: Formalizou a recursão em linguagens imperativas e introduziu mecanismos avançados de transferência de parâmetros (como passagem por valor e por nome).

Uma linguagem pode ser extremamente influente sem dominar o mercado porque sua força pode estar no campo teórico, acadêmico e conceitual. O ALGOL 60 não tinha o patrocínio exclusivo de uma gigante da indústria (como a IBM apoiou o Fortran), não trazia comandos padrão de entrada e saída em sua especificação original (o que atrapalhava a portabilidade industrial imediata) e enfrentou resistência no mercado americano. Apesar disso, suas ideias estruturais foram copiadas por quase todas as linguagens imperativas subsequentes (como Pascal, C, C++ e Java).

7. COBOL foi desenhada para processamento comercial. Mostre como domínio e público influenciaram sua legibilidade, seus registros e sua relação com FLOW-MATIC.

O domínio das aplicações de negócios e o perfil de seus usuários moldaram diretamente o projeto do COBOL:

- Legibilidade e público: O público-alvo incluía administradores, contadores e analistas de sistemas sem formação matemática avançada. Por isso, a linguagem foi projetada para ter uma legibilidade próxima do inglês fluente (prosa), utilizando palavras reservadas completas (como ADD x TO y GIVING z) em vez de símbolos algébricos, além de organizar o programa em "Divisões" bem definidas (IDENTIFICATION, ENVIRONMENT, DATA, PROCEDURE).

- Registros (Data Division): Como o processamento comercial lida com volumes massivos de dados heterogêneos (como arquivos de funcionários, faturas e folhas de pagamento), o COBOL introduziu a DATA DIVISION com estrutura de registros hierárquicos usando níveis numéricos (01, 05, 10), originando o conceito moderno de *structs* e objetos.
- Relação com FLOW-MATIC: COBOL herdou diretamente do FLOW-MATIC (linguagem desenvolvida por Grace Hopper) o conceito de usar comandos em palavras em inglês em vez de códigos matemáticos e a ideia essencial de separar a descrição dos dados da lógica de execução do código.

8. Compare Basic e PL/I como respostas ao desejo de ampliar o acesso ou o alcance da programação. Qual compromisso de projeto aparece em cada caso?

Ambas as linguagens tentaram expandir o uso da programação, mas seguiram caminhos opostos para atingir esse objetivo:

- BASIC (1964):
 - Foco: Ampliar o acesso à computação para iniciantes e estudantes não técnicos em um ambiente de tempo compartilhado (*time-sharing*).
 - Compromisso de projeto: Sacrificou a eficiência e a estrutura de código em favor da simplicidade e facilidade de aprendizado. Abriu mão de verificação avançada de tipos e estruturas de controle complexas para garantir um compilador leve, rápido e amigável.
- PL/I (1965):
 - Foco: Ampliar o alcance da programação, tentando unificar o universo científico (domínio do Fortran) e o universo comercial (domínio do COBOL) em uma única linguagem universal para o IBM System/360.
 - Compromisso de projeto: Sacrificou a simplicidade em favor da universalidade, gerando uma complexidade massiva. A linguagem incluiu tantos recursos de domínios diferentes que se tornou gigante, difícil de aprender por completo, com compiladores pesados e propensa a comportamentos imprevisíveis por conta de conversões implícitas de tipo.

9. APL, SNOBOL e SIMULA 67 seguiram direções distintas. Associe cada linguagem ao seu foco e identifique uma contribuição duradoura de cada uma.

- APL (1962):
 - Foco: Processamento vetorial e matricial de alta densidade para computação matemática expressiva.
 - Contribuição duradoura: Operações com arrays e matrizes inteiras como primitivas nativas da linguagem, influenciando o tratamento moderno de vetores (conceito base para bibliotecas como NumPy).
- SNOBOL (1962):

- Foco: Manipulação de texto, processamento de cadeias de caracteres (*strings*) e casamento de padrões.
- Contribuição duradoura: Conceitos avançados de busca por padrões (*pattern matching*) dinâmicos, que influenciaram o desenvolvimento de linguagens focadas em texto como Perl e o uso de Expressões Regulares.
- SIMULA 67 (1967):
 - Foco: Simulação de sistemas e eventos discretos.
 - Contribuição duradoura: Criação das bases da Programação Orientada a Objetos, introduzindo formalmente os conceitos de *Classes*, *Objetos*, *Corrotinas* e *Herança*.

10. Defina ortogonalidade no projeto de linguagens e use ALGOL 68 para discutir a diferença entre regularidade e simplicidade. Uma linguagem muito ortogonal é automaticamente fácil de usar?

Ortogonalidade é a propriedade de um projeto de linguagem na qual um conjunto reduzido de construções primitivas pode ser combinado de qualquer maneira sem restrições ou exceções arbitrárias. Em uma linguagem ortogonal, o significado de uma construção permanece consistente independentemente do contexto em que é combinada com outras.

No ALGOL 68, a diferença entre regularidade e simplicidade ficou evidente:

- A linguagem levou a ortogonalidade ao extremo em termos de regularidade: praticamente qualquer construção tinha um tipo e retornava um valor, permitindo combinações sintáticas quase infinitas (como atribuir o resultado de um bloco *if* direto dentro de uma expressão matemática).
- No entanto, essa regularidade perfeita não gerou simplicidade. A sintaxe do ALGOL 68 e sua terminologia formal tornaram-se tão abstratas e densas que a linguagem ficou extremamente difícil de ser compreendida, ensinada ou implementada em um compilador real.